05 — EMBEDDED + CONTROLS · 1 OF 2

Arduino Rover

A two-motor rover in C++ that drives set shapes on its own and follows a line on the floor, rebuilt from scratch after its first controller was destroyed.

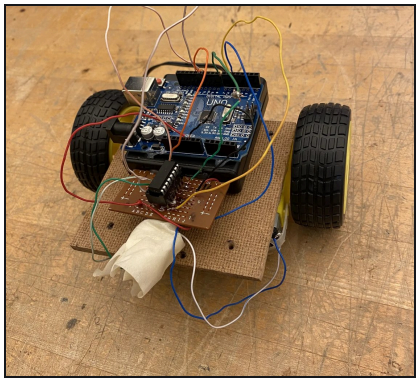


FIG. 5.1 — The rebuilt rover, Jun 2024

CONTEXT	ROLE	YEAR	METHODS
Partner build	Build, soldering, programming, testing	2024	C++ · PWM · IR sensing · soldering

2	<½	0	2
DRIVING MODES: SHAPES AND LINE FOLLOWING	THE ORIGINAL BUILD TIME TO REBUILD FROM BARE PARTS	PARTS REPLACED TO FIX THE DRIFT	DOWNWARD IR SENSORS ON A HAND-SOLDERED BOARD

01 BRIEF

Make a rover drive itself accurately on motors and wheels that were far from ideal. The two motors didn't run at the same speed and the wheels were substandard. Swapping parts wasn't an option, so every problem had to be solved in the build and in the code.

02 CONSTRAINTS

DRIVE	**Two mismatched DC gearmotors, substandard wheels, front caster**
FEEDBACK	**None on the wheels: open loop. Position is only as good as the timing and calibration**
CONTROLLER	**Arduino Uno; pins can't supply motor current directly**
PARTS	**Fixed. Adapt the design, don't replace components**

03 ARCHITECTURE

The Uno is the controller. Its outputs go to a motor driver on a hand-soldered perfboard: an L293D dual H-bridge, which lets the Uno's 5 V logic pins switch the current the motors actually need, in either direction, with PWM on the enable pins setting each motor's speed. Two reflective IR sensors on a second soldered board sit under the front of the chassis, a few millimetres off the floor, reading light versus dark.

Steering is differential: there's no steering mechanism, so the rover turns only by running one wheel faster than the other. That makes it simple to build and very sensitive to any mismatch between the two motors.

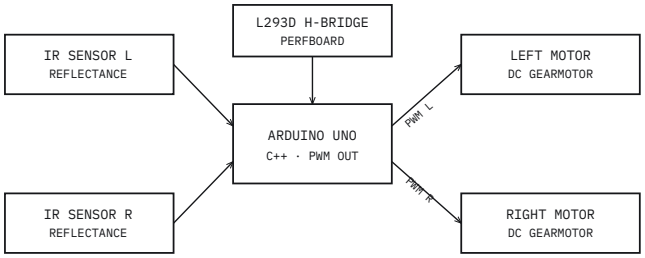


FIG. 5.2 — System block diagram. Driver commands are drawn through the Uno for clarity.

04 TWO MODES, ONE CHASSIS

Shape mode runs timed sequences written in C++: drive straight for a set duration, turn for a set duration, repeat, so the rover traces squares and other geometric shapes on its own.

Line-following mode reads the two IR sensors. When one side sees the line, the code slows that side's motor and the rover steers back over it.

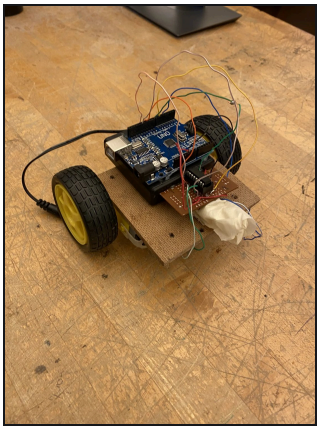
LEFT SENSOR	RIGHT SENSOR	RESPONSE
Floor	Floor	Both motors on: drive straight
Line	Floor	Slow left motor: steer left, back onto the line
Floor	Line	Slow right motor: steer right, back onto the line

Dead board, full restart	FAILURE MODE 5.A
OBSERVED	Partway through the build, the first Arduino was fried. The board was dead, and with it the build: we had to start over.
CAUSE	During testing the rover was hooked up to a battery tester instead of its normal supply. The tester delivered far more power than the circuit was built to take: the motors ran at runaway speed, and the board burned out under the over-voltage.
RESOLVED	Started again from bare parts on a new board. Knowing the build the second time, we finished the full rebuild in **less than half the time** the first attempt took.
PROTECTED	On the rebuild, every connection was checked against the plan and the supply polarity verified before each power-up, and motor power was kept off the logic side.

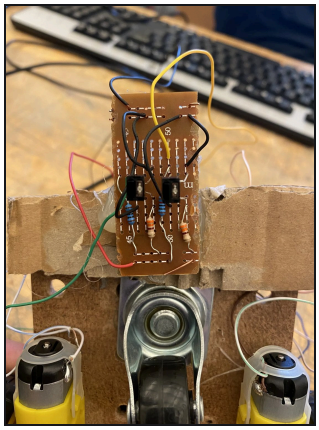
05 WHY THE REBUILD WAS FASTER

The first build was also the learning: working out where each wire went, which order to assemble in, and how to lay out the perfboard. None of that had to be rediscovered. The second build was execution against a design we already understood, which is why it took less than half the time.

It changed how I think about a setback. Losing the hardware didn't lose the work, because the work was the understanding. Being able to rebuild a system quickly matters as much as building it right the first time.



5.3 Uno on the hardboard chassis



5.4 Hand-soldered IR sensor board, underside

Drift	FAILURE MODE 5.B
OBSERVED	Over repeated test runs the rover wandered off its path, and the shapes didn't close.
CAUSE	Equal PWM never gave equal speed. The two motors had different outputs, and gearbox slop and wheel friction added their own bias.
RESOLVED	Calibrated each motor's PWM duty cycle separately, then tuned turn timing, acceleration delays and the movement sequences run by run until both sides matched.
VERIFIED	Shapes came out consistent across repeated runs, on the same motors and wheels it started with.

06 WHY SMALL MISMATCHES MATTER

With differential steering, the rover's turning rate is set by the difference between its wheel speeds divided by the distance between the wheels. Any difference at all becomes a steady turn, and with no encoders to notice it, the error builds up for as long as the rover drives. A mismatch too small to see on the bench becomes a square that doesn't close.

Turn rate	$\omega = (v_R - v_L) / \text{track width}$
Heading error	$\Delta\theta = \omega \cdot t$, growing every second
Correction applied	separate PWM duty per motor

Calibration doesn't remove the cause, it cancels it: each motor gets the duty cycle that makes its real speed match the other's. That's the limit of an open-loop system, and the reason encoders are first on the Rev. B list.

07 OUTCOME

A rover that draws its shapes consistently and follows a line, on the same imperfect motors and wheels it started with, delivered despite losing the first build entirely.

> *When the parts aren't ideal, fix it in the design and the code. When the hardware is lost, the understanding isn't.*

08 REV. B

- Wheel encoders for closed-loop speed control instead of fixed PWM offsets.
- PID steering in line-following mode for smoother, faster tracking.
- A fused, separate motor supply to protect the logic board.